

01 — Monitoring Overview: What to Watch in Production PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Why Monitoring Matters](#)
 3. [The Three Monitoring Layers](#)
 4. [What to Monitor](#)
 5. [OS Layer Signals](#)
 6. [PostgreSQL Layer Signals](#)
 7. [Application Layer Signals](#)
 8. [Alert Thresholds Reference](#)
 9. [Monitoring Strategy and Tooling](#)
 10. [SQL Monitoring Queries](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Interview Questions and Answers](#)
 14. [Cross-References](#)
-

Learning Objectives

By the end of this section you will be able to:

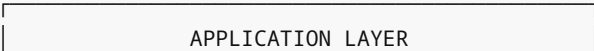
- Identify the three layers of a PostgreSQL monitoring stack and explain what each layer contributes
 - Enumerate the key signals (connections, locks, query performance, replication lag, disk, memory, bgwriter, autovacuum) and explain why each matters
 - Write SQL queries that surface the most critical health indicators from PostgreSQL catalog views
 - Define sensible alert thresholds for production environments and justify the values
 - Distinguish between monitoring (continuous observation) and alerting (threshold-triggered notification)
-

Why Monitoring Matters

A PostgreSQL database can degrade silently. Table bloat accumulates over weeks. Replication lag climbs during a batch load. A single long-running transaction blocks autovacuum and causes dead-tuple accumulation that eventually slows every query touching that table. None of these failures announce themselves loudly at first; they are discovered only when a user reports slowness or, worse, when the application crashes.

Effective monitoring converts invisible problems into visible signals before they become outages. The goal is not to collect every possible metric — it is to collect the *right* metrics at the *right* granularity so that the on-call engineer has enough context to diagnose the problem within minutes, not hours.

The Three Monitoring Layers



APPLICATION LAYER

query latency p50/p95/p99, error rates, connection pool saturation, slow queries
POSTGRESQL LAYER pg_stat_* views, wait events, autovacuum, checkpoint pressure, replication lag
OS LAYER CPU, memory, disk I/O, network, swap, file descriptors, kernel parameters

Each layer provides a different perspective:

Layer	Answers	Tools
OS	"Is the machine struggling?"	sar, iostat, vmstat, node_exporter
PostgreSQL	"Is the database engine struggling?"	pg_stat_* views, pg_logs, postgres_exporter
Application	"Is the user experiencing degradation?"	APM (Datadog, New Relic), query logs, connection pool metrics

Correlating across layers is essential. High I/O wait at the OS level combined with a low cache-hit ratio in `pg_stat_database` and many sequential scans in `pg_stat_user_tables` points to missing indexes. High CPU with many rows-examined in slow query logs points to inefficient query plans.

What to Monitor

1. Connections

PostgreSQL has a hard limit on connections (`max_connections`). When that limit is reached, every new connection attempt fails with `FATAL: sorry, too many clients already`. Monitoring connections prevents this scenario.

Key metrics:

- numbackends per database (from pg_stat_database)
- State breakdown: active , idle , idle in transaction , idle in transaction (aborted)
- Age of oldest idle in transaction session (seconds since state_change)
- Connections per application_name

Why idle in transaction is dangerous: A session in this state holds an open transaction. It pins the transaction ID, preventing autovacuum from cleaning dead rows past that XID. It may hold row-level locks that block writers. A single forgotten `idle in transaction` connection can cascade into table bloat and lock contention.

2. Locks

PostgreSQL implements MVCC but still uses heavyweight locks for DDL and certain DML. Lock waits are one of the most common sources of production incidents.

Key metrics:

- Number of lock waits currently active (`pg_locks` + `pg_stat_activity`)
- Age of the oldest lock wait (seconds)
- Blocking query identification (the query holding the lock that others are waiting for)
- Lock type distribution (`AccessShareLock`, `RowExclusiveLock`, `AccessExclusiveLock`)

Why AccessExclusiveLock is dangerous: `ALTER TABLE` , `VACUUM FULL` , `CLUSTER` , and certain index operations acquire `AccessExclusiveLock`. While this lock is waiting to be granted, it blocks all other lock requests on the same object — including simple `SELECT` statements. This is because PostgreSQL queues lock requests in order, so a waiting DDL statement blocks all subsequent readers.

3. Query Performance

Slow queries consume CPU, I/O, memory, and lock resources. Identifying them quickly is the cornerstone of performance management.

Key metrics:

- Queries slower than threshold from `pg_stat_activity` (currently executing)
- Historical slow queries from `pg_stat_statements` (if enabled)
- Average, p95, p99 execution times per query fingerprint
- Queries with high `rows_examined` / `rows_returned` ratio (full scans)
- Plans with sequential scans on large tables

4. Replication Lag

In a primary-replica setup, replica lag is the number of bytes (or seconds) by which the replica is behind the primary. Applications reading from replicas may see stale data. If the primary fails and the replica has lag, some data may not be promoted.

Key metrics:

- `pg_stat_replication.write_lag` , `flush_lag` , `replay_lag` (time intervals, PG 10+)
- `sent_lsn` - `replay_lsn` (byte lag, calculated from LSN positions)
- `pg_replication_slots` for inactive slots (can cause unbounded WAL accumulation)

5. Disk Usage

PostgreSQL data directories can grow due to table growth, bloat, WAL accumulation, and index growth. A full disk causes immediate database shutdown.

Key metrics:

- Data directory free space (OS-level, `node_exporter`)
- WAL directory size (`pg_wal/` or `pg_xlog/` pre-PG10)
- Table sizes (`pg_total_relation_size`)
- Index sizes
- Tablespace utilization
- Temp file usage (`pg_stat_database.temp_bytes`)

6. Memory

PostgreSQL uses shared memory (`shared_buffers`) for its buffer cache and process-local memory (`work_mem`) for sort and hash operations. Insufficient memory leads to disk-based sorts and hash joins, which are orders of magnitude slower.

Key metrics:

- Buffer cache hit ratio (`pg_stat_database.blks_hit / (blks_hit + blks_read)`)
- Shared buffer dirty pages waiting to be written (bgwriter metrics)
- OS-level: total memory, available memory, swap usage
- `work_mem` overflow to temp files (`pg_stat_database.temp_files` , `temp_bytes`)

7. Background Writer (bgwriter)

The background writer (bgwriter) proactively writes dirty shared buffers to disk so that when a backend needs a clean buffer, it doesn't have to wait for a write. If the bgwriter cannot keep up, backends write dirty buffers themselves — a condition called "backend writes" — which causes query latency spikes.

Key metrics from `pg_stat_bgwriter` :

- `buffers_clean` (written by bgwriter — good)
- `buffers_backend` (written by backends — indicates bgwriter is behind)
- `maxwritten_clean` (bgwriter hit its round limit — increase `bgwriter_lru_maxpages`)
- `buffers_checkpoint` (written during checkpoints)
- `checkpoint_req` vs `checkpoint_timed` (too many requested checkpoints = `max_wal_size` too small)

8. Autovacuum

Autovacuum reclaims dead tuple space and freezes old transaction IDs to prevent XID wraparound. If autovacuum cannot keep up, tables bloat, queries slow down, and eventually PostgreSQL may be forced to shutdown to prevent wraparound.

Key metrics:

- `n_dead_tup` and `n_live_tup` from `pg_stat_user_tables` (dead tuple ratio)
- `last_autovacuum` and `last_autoanalyze` timestamps
- Tables where autovacuum is running: `pg_stat_activity` where `query ~ 'autovacuum'`
- `age(relfrozenxid)` from `pg_class` (XID age — must stay below 2 billion)
- `age(datfrozenxid)` from `pg_database`
- Autovacuum cancellations (logged when `autovacuum_vacuum_cost_delay` throttling causes timeout)

OS Layer Signals

Signal	What it Means	Tool	Alert Threshold
CPU utilization	High CPU may indicate inefficient queries, missing indexes, or sorts	top, sar, node_exporter	>80% sustained 5 min
I/O wait (%iowait)	Processes waiting on disk; often means shared_buffers too small or unindexed queries	iostat, sar	>20% sustained
Disk free space	Full disk = immediate crash	df, node_exporter	<20% free (warn), <10% free (critical)
Memory available	OOM kill risk if memory exhausted	free, vmstat	<10% available

Swap usage	Active swapping indicates memory pressure	vmstat, sar	>0 (warn), >10% (critical)
Open file descriptors	PostgreSQL opens FDs per connection	/proc/sys/fs/file-nr	>80% of ulimit
Network throughput	Replication and client traffic	sar -n DEV	per-environment baseline

PostgreSQL Layer Signals

Signal	Source View	Key Column	Alert Threshold
Active connections	pg_stat_database	numbackends	>80% of max_connections
Idle in transaction	pg_stat_activity	state + age	Any session >5 minutes
Lock waits	pg_locks	granted=false	Any wait >30 seconds
Cache hit ratio	pg_stat_database	blks_hit/(blks_hit+blks_read)	<0.99 (warn), <0.95 (critical)
Temp files	pg_stat_database	temp_files	>100/hour
Dead tuple ratio	pg_stat_user_tables	n_dead_tup/n_live_tup	>20% per table
XID age	pg_class	age(relfrozenxid)	>1.5B (warn), >1.9B (critical)
Replication lag	pg_stat_replication	replay_lag	>30s (warn), >5min (critical)
Backend writes	pg_stat_bgwriter	buffers_backend	>10% of total buffers written
Checkpoint frequency	pg_stat_bgwriter	checkpoint_req	>1 per minute
Long-running queries	pg_stat_activity	now()-query_start	>5 min (warn), >30 min (critical)

Application Layer Signals

Signal	Description	Alert Threshold
Query latency p99	99th percentile response time	>2x baseline
Error rate	HTTP 500s or DB connection errors	>0.1% request rate
Connection pool wait time	Time waiting for a pool slot	>100ms
Connection pool saturation	Pool slots in use / total	>90%

Transaction throughput (TPS)	Transactions per second	<50% of baseline (drop)
Slow query count	Queries exceeding SLA threshold	>10 per minute

Alert Thresholds Reference

CRITICAL – Page on-call immediately

- └─ Disk free space < 10%
- └─ pg_wal directory growing unbounded (inactive replication slot)
- └─ age(datfrozenxid) > 1.9 billion
- └─ Replication lag > 10 minutes
- └─ All connection slots exhausted
- └─ PostgreSQL process not running
- └─ Checkpoint failure logged

WARNING – Investigate within 1 hour

- └─ Disk free space < 20%
- └─ Cache hit ratio < 0.99
- └─ Dead tuple ratio > 20% on any large table
- └─ age(datfrozenxid) > 1.5 billion
- └─ Replication lag > 60 seconds
- └─ Connection count > 80% of max_connections
- └─ Any session idle in transaction > 5 minutes
- └─ Backend writes > 10% of total writes
- └─ Checkpoint requested > 1/minute

INFO – Review during business hours

- └─ Temp file usage trending up
- └─ autovacuum not running on tables > 1 hour
- └─ New sequential scans on tables > 1M rows
- └─ work_mem overflow to disk

SQL Monitoring Queries

Q1: Connection Summary by State

```
-- Overview of all connection states across databases
SELECT
    datname,
    state,
    COUNT(*) AS conn_count,
    MAX(EXTRACT(EPOCH FROM (now() - state_change)))::INT AS max_age_seconds
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
GROUP BY datname, state
ORDER BY datname, conn_count DESC;
```

Q2: Total Connections vs Max Connections

```
-- How close are we to the connection limit?
SELECT
    COUNT(*) AS current_connections,
    current_setting('max_connections')::INT AS max_connections,
    ROUND(COUNT(*) * 100.0 / current_setting('max_connections')::INT, 1) AS
pct_used,
    current_setting('max_connections')::INT - COUNT(*) AS slots_remaining
FROM pg_stat_activity
WHERE pid <> pg_backend_pid();
```

Q3: Idle in Transaction Sessions

```
-- Find sessions stuck in open transactions
SELECT
    pid,
    username,
    application_name,
    client_addr,
    state,
    EXTRACT(EPOCH FROM (now() - state_change))::INT AS idle_seconds,
    LEFT(query, 100) AS last_query
FROM pg_stat_activity
WHERE state IN ('idle in transaction', 'idle in transaction (aborted)')
ORDER BY idle_seconds DESC;
```

Q4: Long-Running Active Queries

```
-- Active queries running longer than 60 seconds
SELECT
    pid,
    username,
    datname,
    EXTRACT(EPOCH FROM (now() - query_start))::INT AS duration_seconds,
    wait_event_type,
    wait_event,
    LEFT(query, 200) AS query
FROM pg_stat_activity
WHERE state = 'active'
    AND query_start < now() - INTERVAL '60 seconds'
    AND query NOT LIKE '%pg_stat_activity%'
ORDER BY duration_seconds DESC;
```

Q5: Cache Hit Ratio per Database

```
-- Buffer cache effectiveness - should be > 0.99
SELECT
    datname,
    blks_hit,
```

```

    blks_read,
    CASE
        WHEN blks_hit + blks_read = 0 THEN NULL
        ELSE ROUND(blks_hit * 100.0 / (blks_hit + blks_read), 4)
    END AS cache_hit_ratio,
    temp_files,
    temp_bytes / (1024*1024) AS temp_mb
FROM pg_stat_database
WHERE datname NOT IN ('template0', 'template1')
ORDER BY blks_read DESC;

```

Q6: Lock Waits — Who Is Blocking Whom

```

-- Find blocking locks and what they are blocking
SELECT
    blocked.pid AS blocked_pid,
    blocked.username AS blocked_user,
    blocked.query AS blocked_query,
    blocking.pid AS blocking_pid,
    blocking.username AS blocking_user,
    blocking.query AS blocking_query,
    EXTRACT(EPOCH FROM (now() - blocked.query_start))::INT AS wait_seconds
FROM pg_stat_activity AS blocked
JOIN pg_locks AS bl ON bl.pid = blocked.pid AND NOT bl.granted
JOIN pg_locks AS kl ON kl.transactionid = bl.transactionid AND kl.granted
JOIN pg_stat_activity AS blocking ON blocking.pid = kl.pid
ORDER BY wait_seconds DESC;

```

Q7: Replication Lag

```

-- Replication status and lag per standby
SELECT
    application_name,
    client_addr,
    state,
    sync_state,
    write_lag,
    flush_lag,
    replay_lag,
    pg_wal_lsn_diff(sent_lsn, replay_lsn) AS byte_lag
FROM pg_stat_replication
ORDER BY replay_lag DESC NULLS LAST;

```

Q8: Dead Tuple Accumulation by Table

```

-- Tables with high dead tuple ratios – autovacuum candidates
SELECT
    schemaname,

```

```

relname AS table_name,
n_live_tup,
n_dead_tup,
CASE
    WHEN n_live_tup + n_dead_tup = 0 THEN 0
    ELSE ROUND(n_dead_tup * 100.0 / (n_live_tup + n_dead_tup), 2)
END AS dead_pct,
last_vacuum,
last_autovacuum,
last_analyze,
last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 20;

```

Q9: XID Age — Wraparound Risk

```

-- Transaction ID age per table – must stay below 2 billion
SELECT
    schemaname,
    relname,
    age(relfrozenxid) AS xid_age,
    pg_size_pretty(pg_total_relation_size(oid)) AS total_size,
    ROUND(age(relfrozenxid) * 100.0 / 2147483647, 2) AS pct_of_max
FROM pg_class
WHERE relkind = 'r'
    AND schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY xid_age DESC
LIMIT 20;

```

Q10: bgwriter Efficiency

```

-- Background writer statistics
SELECT
    checkpoints_timed,
    checkpoints_req,
    ROUND(checkpoints_req * 100.0 / NULLIF(checkpoints_timed + checkpoints_req, 0),
2) AS pct_requested,
    buffers_checkpoint,
    buffers_clean,
    maxwritten_clean,
    buffers_backend,
    ROUND(buffers_backend * 100.0 / NULLIF(buffers_checkpoint + buffers_clean +
buffers_backend, 0), 2) AS pct_backend_writes,
    stats_reset
FROM pg_stat_bgwriter;

```

Q11: Database-Level Overview

```
-- One-row-per-database health summary
SELECT
    datname,
    numbackends,
    xact_commit,
    xact_rollback,
    ROUND(xact_rollback * 100.0 / NULLIF(xact_commit + xact_rollback, 0), 2) AS
rollback_pct,
    deadlocks,
    conflicts,
    blks_hit,
    blks_read,
    ROUND(blks_hit * 100.0 / NULLIF(blks_hit + blks_read, 0), 4) AS cache_hit_ratio,
    temp_files,
    pg_size_pretty(temp_bytes) AS temp_size
FROM pg_stat_database
WHERE datname NOT IN ('template0', 'template1')
ORDER BY numbackends DESC;
```

Q12: Autovacuum Activity

```
-- Currently running autovacuum workers
SELECT
    pid,
    datname,
    username,
    EXTRACT(EPOCH FROM (now() - query_start))::INT AS running_seconds,
    LEFT(query, 150) AS autovacuum_query
FROM pg_stat_activity
WHERE query LIKE 'autovacuum:%'
ORDER BY running_seconds DESC;
```

Q13: Tables Not Vacuumed Recently

```
-- Tables that have not been vacuumed in the last 24 hours
SELECT
    schemaname,
    relname,
    n_dead_tup,
    n_live_tup,
    COALESCE(last_autovacuum, last_vacuum) AS last_vacuued,
    EXTRACT(HOURS FROM (now() - COALESCE(last_autovacuum, last_vacuum))) AS
hours_since_vacuum
FROM pg_stat_user_tables
WHERE COALESCE(last_autovacuum, last_vacuum) < now() - INTERVAL '24 hours'
    OR (last_autovacuum IS NULL AND last_vacuum IS NULL)
ORDER BY n_dead_tup DESC;
```

Common Mistakes

1. **Monitoring only from the application side.** Application-level query latency can look fine while table bloat accumulates — until it suddenly doesn't. Always include database-internal metrics.
2. **Setting `max_connections` too high.** Each connection consumes ~5–10 MB of memory. Setting `max_connections = 1000` without a connection pooler causes memory pressure before connections are exhausted.
3. **Ignoring `idle in transaction` connections.** Most monitoring dashboards show total connections, not broken down by state. A database with 50 `idle in transaction` connections is in a worse state than one with 200 active ones.
4. **Not monitoring replication slots.** An inactive replication slot prevents WAL deletion. The `pg_wal/` directory can fill the disk silently over hours.
5. **Alert fatigue from thresholds set too low.** If you alert on cache hit ratio < 0.9999%, you will page the on-call engineer hundreds of times per night. Calibrate thresholds against your actual baseline.
6. **No alerting on `age(datfrozenxid)`.** XID wraparound is a database-stopping event. Many teams discover they are within 100M transactions of the limit only when PostgreSQL starts logging warnings.
7. **Polling `pg_stat_activity` too infrequently.** A query that runs for 5 minutes and holds a lock that blocks writes for 4 minutes will not be visible if you only poll every 5 minutes.

Best Practices

- **Collect metrics at 15-second intervals** for `pg_stat_activity` and locks; 60-second intervals for slower-changing metrics like table sizes.
- **Use `pg_stat_statements`** for query-level aggregation. Enable it with `shared_preload_libraries = 'pg_stat_statements'` in `postgresql.conf`.
- **Establish baselines before setting thresholds.** Run 2–4 weeks of monitoring to understand your normal TPS, cache hit ratio, and connection counts before setting alert thresholds.
- **Monitor the monitoring.** Ensure your monitoring agent (e.g., `postgres_exporter`) is itself healthy and collecting data. A gap in metrics during an incident is as bad as no monitoring.
- **Create a "war room" dashboard** that shows the 10–15 most critical signals on a single screen for incident response.
- **Keep a `pg_stat_bgwriter` reset schedule.** Reset stats with `SELECT pg_stat_reset()` or `SELECT pg_stat_reset_shared('bgwriter')` at known intervals so cumulative counters are meaningful.
- **Alert on WAL directory growth rate**, not just size. A rapidly growing `pg_wal/` often precedes a disk-full incident.

Interview Questions and Answers

Q1: What are the three layers of PostgreSQL monitoring and what does each layer contribute?

A: The three layers are OS, PostgreSQL engine, and application. The OS layer reveals resource constraints (CPU, memory, disk I/O, swap). The PostgreSQL layer exposes internal engine state via `pg_stat_*` views — wait events, autovacuum progress, replication lag, buffer cache usage. The application layer shows the user-

facing impact — query latency percentiles, error rates, connection pool saturation. Correlating across all three is required for accurate root-cause analysis.

Q2: Why are `idle in transaction` connections particularly dangerous?

A: They hold an open transaction, which prevents autovacuum from vacuuming dead rows with XID greater than the transaction's snapshot. They may also hold row-level or object-level locks that block concurrent writers. A single session idle in transaction for hours can cause significant table bloat and lock-induced outages. Production systems should set `idle_in_transaction_session_timeout` to automatically terminate these sessions.

Q3: What does the `buffers_backend` counter in `pg_stat_bgwriter` indicate and why is it a warning sign?

A: `buffers_backend` counts the number of dirty shared buffers written directly by backend processes rather than by the bgwriter or checkpoint. When a backend needs a free buffer and none are available, it writes a dirty buffer itself. This blocks the backend query for the duration of the write, causing latency spikes. A high `buffers_backend` relative to total writes indicates the bgwriter cannot keep up — either `bgwriter_lru_maxpages` is too low, `bgwriter_lru_multiplier` needs tuning, or `shared_buffers` is undersized.

Q4: What is XID wraparound and how do you detect it before it becomes an emergency?

A: PostgreSQL uses 32-bit transaction IDs (XIDs). With $2^{31} \approx 2.1$ billion transactions, PostgreSQL uses a circular numbering scheme. If a table's `relfrozenxid` is more than 2 billion transactions behind the current XID, PostgreSQL cannot determine whether rows in that table are visible, so it refuses to serve queries against that table (and eventually shuts down entirely). Detection: query `age(relfrozenxid) from pg_class` and `age(datfrozenxid) from pg_database`. Alert when age exceeds 1.5 billion; escalate at 1.9 billion. Resolution is emergency `VACUUM FREEZE` on affected tables.

Q5: How do you identify which query is blocking other queries in PostgreSQL?

A: Join `pg_locks` with `pg_stat_activity` twice — once for blocked sessions (where `granted = false`) and once for blocking sessions (where the same lock is `granted = true`). The query in the blocking session is what needs to be examined or terminated. Modern PostgreSQL (9.6+) also provides `pg_blocking_pids(pid)` as a convenient function.

Q6: What is a safe alert threshold for cache hit ratio and what should you do if it drops below that threshold?

A: A cache hit ratio below 0.99 (99%) is a warning; below 0.95 is critical for OLTP workloads. When the ratio drops: first check `pg_stat_user_tables` for tables with many sequential scans — these are likely causing cold reads. Check if `shared_buffers` is appropriately sized (typically 25% of RAM). Look for recently added large table scans or full-table sequential scans caused by missing indexes. Consider increasing `shared_buffers` if memory allows, or adding `pg_prewarm` to load hot data after restarts.

Q7: Why is monitoring replication slots critically important?

A: An inactive or lagging replication slot causes PostgreSQL to retain all WAL segments generated since the slot's last confirmed LSN. If the consumer of the slot (e.g., a logical replication subscriber or a CDC tool like Debezium) falls far behind or stops entirely, the `pg_wal/` directory can grow without bound until the disk is full. This is one of the most common causes of unexpected disk-full incidents in PostgreSQL. Alert on `pg_replication_slots` where `active = false` and `pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)` exceeds a threshold.

Q8: What is the difference between a `checkpoint_timed` and a `checkpoint_req` in `pg_stat_bgwriter` ?

A: A `checkpoint_timed` occurs at the interval defined by `checkpoint_timeout` (default 5 minutes). A `checkpoint_req` is triggered because the amount of WAL generated since the last checkpoint exceeded `max_wal_size` . A high ratio of requested to timed checkpoints means `max_wal_size` is too small for your write rate, causing frequent checkpoints that stress I/O. The fix is to increase `max_wal_size` (at the cost of longer recovery time) or reduce write rate.

Cross-References

- `02_pg_stat_views.md` — detailed reference for every `pg_stat_*` view used in this section
- `04_observability_stack.md` — Prometheus, Grafana, and PgHero setup to operationalize these metrics
- `05_capacity_planning.md` — using monitoring data to project growth
- `07_incident_response.md` — runbooks for each alert type described here
- `08_performance_tuning_config.md` — tuning `shared_buffers` , `bgwriter` , and `autovacuum` based on monitoring data
- `09_troubleshooting_guide.md` — systematic investigation using these queries